

AI-STUDYBUDDY: MASTER TECHNICAL DOCUMENTATION PACKAGE

Project Name: AI-STUDYBUDDY

Team ID: SWTID-2026-7347

Team Size: 4

Team Leader: HIMESH ROSHAN M J

Team Members: HARISH M, JAGATHEESHWARAN K, KARTHIKEYAN R

DOCUMENT INDEX

1. Document 1: Problem Statement & Scope Definition
2. Document 2: Software Requirement Specification (SRS)
3. Document 3: System Architecture & Design Document
4. Document 4: Database Schema & Entity-Relationship Documentation
5. Document 5: REST API Specification Document
6. Document 6: Module Responsibility & Task Allocation Matrix
7. Document 7: Master Test Plan & Test Cases
8. Document 8: Deployment, Setup & Environment Guide
9. Document 9: Video Demo Script & Screen Recording Guide
10. Document 10: Final Comprehensive Project Report

DOCUMENT 1: PROBLEM STATEMENT & SCOPE DEFINITION

1.1 Executive Summary

Students across higher education face massive information overload. Lecture notes, textbooks, and supplemental materials are often fragmented and dense. Synthesizing key concepts, creating self-assessment quizzes, and building structured study plans require significant manual effort. AI-STUDYBUDDY addresses this by providing an automated, RESTful backend engine that processes raw study material and leverages Large Language Models (LLMs) to automatically generate summaries, interactive quizzes, flashcards, and personalized study schedules.

1.2 Problem Statement

Traditional study workflows present three main bottlenecks:

1. **Time-Intensive Content Synthesis:** Extracting core principles from dense text files manually takes hours.
2. **Lack of Automated Self-Assessment:** Students rarely have immediate access to targeted practice quizzes based directly on their specific syllabus materials.
3. **Inflexible Planning:** Converting learning objectives into realistic daily study timelines is difficult without structured scheduling tools.

1.3 Project Objectives

- Implement a secure MERN-stack backend using Node.js, Express.js, MongoDB, and JSON Web Tokens (JWT).
- Provide robust document ingestion supporting text file parsing and metadata persistent storage using Multer and MongoDB.
- Integrate Google Gemini API (gemini-2.5-flash) to deliver real-time document summarization, flashcard generation, quiz synthesis, and study schedule planning.
- Secure all user data and AI generation endpoints using HTTP-only Authorization Bearer headers and bcryptjs password encryption.

1.4 Scope & Boundaries

- **In-Scope:**
 - User Registration and Authentication via JWT.
 - Single-file upload processing (multipart/form-data).
 - MongoDB document storage for metadata and raw text.
 - AI endpoints: Summarization, Flashcard Generation, Quiz Generation, Study Plan Synthesis.
 - Comprehensive error handling and asynchronous route middleware.
- **Out-of-Scope:**
 - Native Mobile Client Applications.
 - Direct payment gateway integration.

DOCUMENT 2: SOFTWARE REQUIREMENT SPECIFICATION (SRS)

2.1 Functional Requirements

Module 1: User Authentication & Authorization (Harish M)

- **FR-1.1:** System shall allow new users to register with name, email, and password.
- **FR-1.2:** System shall hash passwords using bcryptjs before persisting to MongoDB.
- **FR-1.3:** System shall issue a signed JSON Web Token upon successful credential validation.
- **FR-1.4:** Protected routes shall enforce authentication via Authorization: Bearer <TOKEN>.

Module 2: Material Storage & Management (Jagatheeshwaran K)

- **FR-2.1:** System shall ingest user-uploaded documents via Multer middleware.

- **FR-2.2:** System shall extract raw textual content and store metadata (owner ID, filename, filepath, upload timestamp) in MongoDB.
- **FR-2.3:** System shall allow users to retrieve uploaded materials by unique Document Object ID.
- **FR-2.4:** System shall support material deletion and maintain clean storage state.

Module 3: AI Core Processing & Intelligence (Karthikeyan R)

- **FR-3.1:** System shall accept document IDs and pass stored text into Google Gemini API prompts.
- **FR-3.2:** System shall generate bulleted summaries emphasizing core academic principles.
- **FR-3.3:** System shall produce structured Multiple Choice Questions (MCQs) complete with options and correct answers.
- **FR-3.4:** System shall generate tailored daily study plans based on goal inputs, target duration, and study hours.

2.2 Non-Functional Requirements

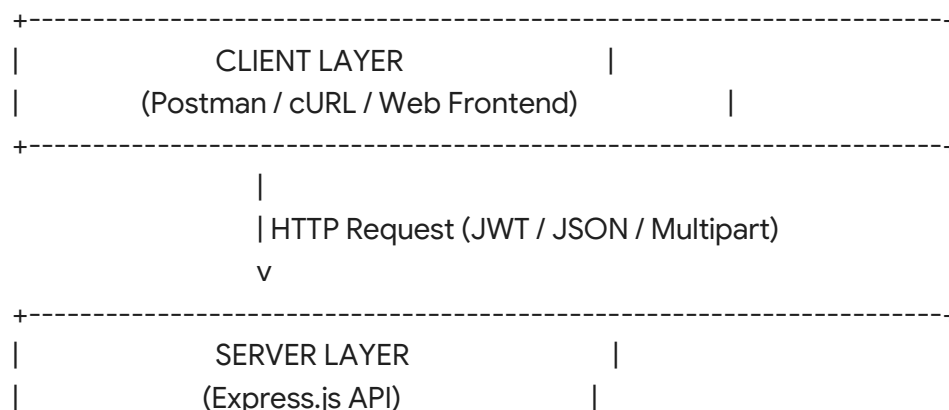
- **NFR-1 Performance:** API endpoints shall respond within 2.5 seconds for AI operations under normal network conditions.
- **NFR-2 Security:** Passwords must be hashed with a minimum cost factor of 10. API secrets must remain in .env configuration files.
- **NFR-3 Reliability:** The AI module must implement fallback model invocation to maintain uptime during high traffic.
- **NFR-4 Scalability:** Express router structure must support horizontal scaling with stateless JWT verification.

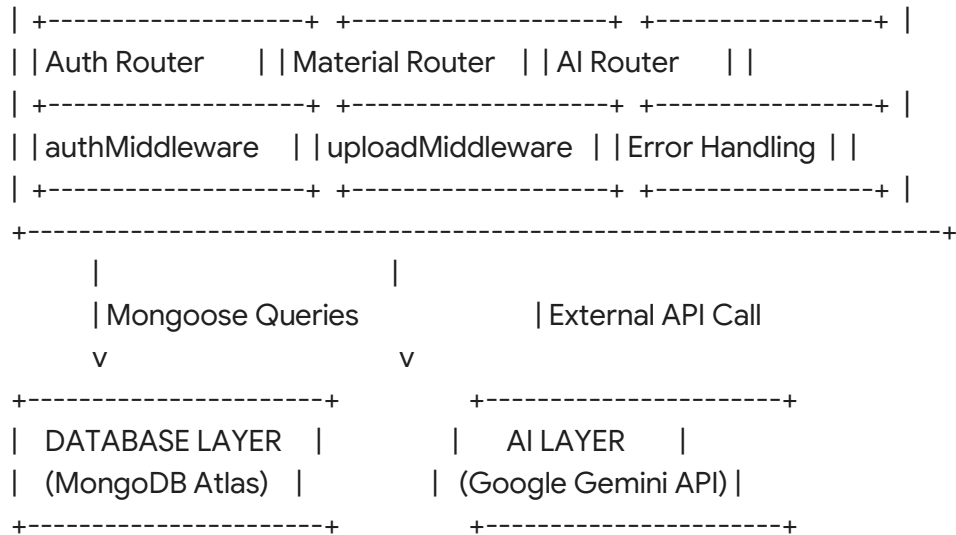
DOCUMENT 3: SYSTEM ARCHITECTURE & DESIGN

DOCUMENT

3.1 High-Level Architecture

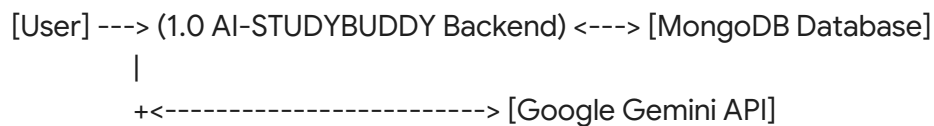
AI-STUDYBUDDY follows a clean, layered RESTful Architecture pattern:



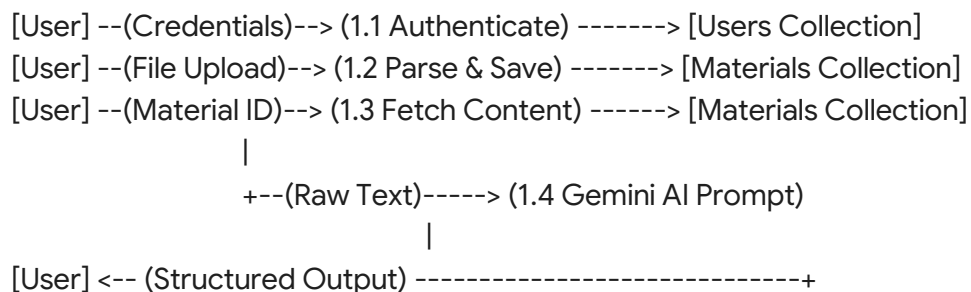


3.2 Data Flow Diagrams (DFD)

Level 0 DFD



Level 1 DFD



DOCUMENT 4: DATABASE SCHEMA & ENTITY-RELATIONSHIP DOCUMENTATION

4.1 Data Models

1. User Schema (User.js)

```
const userSchema = new mongoose.Schema({
```

```

name: { type: String, required: true },
email: { type: String, required: true, unique: true },
password: { type: String, required: true }
}, { timestamps: true });

```

2. Material Schema (Material.js)

```

const materialSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  title: { type: String, required: true },
  content: { type: String, required: true },
  fileUrl: { type: String }
}, { timestamps: true });

```

4.2 Entity-Relationship Structure

- User to Material:** 1-to-Many ($1 : N$). A single registered user can upload multiple study materials. Each material belongs to exactly one user.

DOCUMENT 5: REST API SPECIFICATION DOCUMENT

5.1 Endpoints Overview

Category	Endpoint	Method	Auth Required	Description
Auth	/api/auth/register	POST	No	Creates new user account
Auth	/api/auth/login	POST	No	Validates credentials & returns JWT
Material	/api/materials/upload	POST	Yes	Uploads material file (multipart)
Material	/api/materials	GET	Yes	Retrieves all user materials

Material	/api/materials/:id	GET	Yes	Retrieves single material by ID
AI	/api/materials/:id/summarize	POST	Yes	Generates bulleted text summary
AI	/api/materials/:id/quiz	POST	Yes	Generates structured MCQs
AI	/api/materials/:id/study-plan	POST	Yes	Generates custom study timeline

5.2 API Payloads & Example Responses

POST /api/materials/:id/quiz

- **Request Headers:**
 - Authorization: Bearer <JWT_TOKEN>
 - Content-Type: application/json
- **Request Body:**

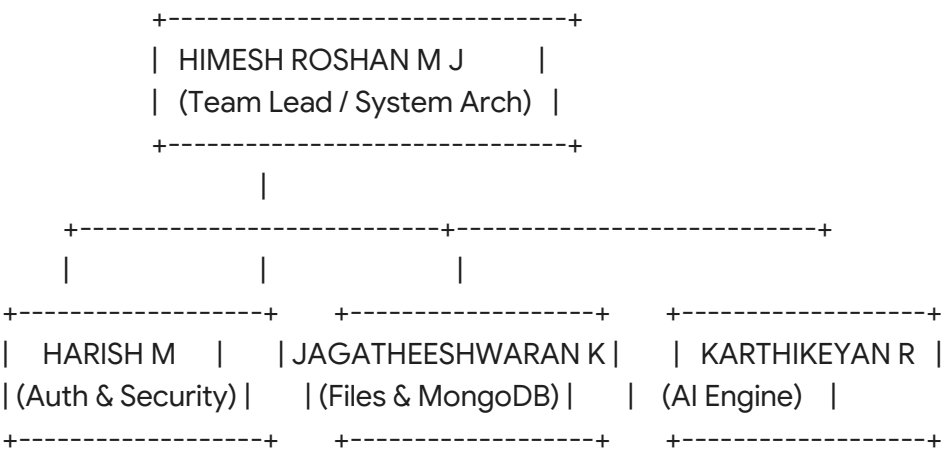
```
{
  "count": 3
}
```
- **Success Response (200 OK):**

```
{
  "quiz": [
    {
      "question": "What principle governs stack operations?",
      "options": ["FIFO", "LIFO", "FILO", "Random"],
      "answer": "LIFO"
    }
  ]
}
```

DOCUMENT 6: MODULE RESPONSIBILITY & TASK

ALLOCATION MATRIX

6.1 Team Composition & Roles



6.2 Responsibility Allocation Matrix (RACI)

Milestone / Task	Himesh Roshan M J	Harish M	Jagatheeshwaran K	Karthikeyan R
System Architecture & Project Setup	Accountable	Consulted	Consulted	Consulted
Authentication & JWT Security	Reviewed	Responsible	Support	Support
Multer Ingestion & Mongo Schemas	Reviewed	Support	Responsible	Support
Google Gemini API Core Integration	Reviewed	Support	Support	Responsible
Postman End-to-End Verification	Responsible	Support	Support	Support

DOCUMENT 7: MASTER TEST PLAN & TEST CASES

7.1 Testing Strategy

Testing was executed systematically across Unit, Integration, and System levels using Postman API testing suites.

7.2 Execution Test Cases

Test ID	Module	Inputs	Expected Result	Actual Result	Status
TC-01	Auth	Register valid user payload	201 Created with User object	User created successfully	PASS
TC-02	Auth	Login with incorrect password	401 Unauthorized response	Returns Invalid Credentials	PASS
TC-03	Material	Upload .txt file without JWT	401 Unauthorized token error	Access Denied	PASS
TC-04	Material	Upload test_material.txt	201 Created with Object_id	Document stored in Mongo	PASS
TC-05	AI Engine	POST /summarize on valid ID	200 OK with JSON Summary	Summary returned	PASS
TC-06	AI Engine	POST /quiz with count = 3	200 OK with 3 MCQs	Array of 3 MCQs generated	PASS

DOCUMENT 8: DEPLOYMENT, SETUP & ENVIRONMENT GUIDE

8.1 Prerequisites

- Node.js v18.x or higher
- MongoDB Atlas connection string
- Google Gemini API Key

8.2 Environment Configuration (.env)

Create a .env file in the project root:

```
PORT=5000
```

```
MONGO_URI=mongodb+srv://<username>:<password>@cluster0.mongodb.net/studybuddy
```

```
JWT_SECRET=super_secret_jwt_key_2026
```

```
GEMINI_API_KEY=AlzaSyYourGeminiApiKeyHere
```

8.3 Installation Commands

```
# Clone and enter project directory
cd AI-STUDYBUDDY
```

```
# Install dependencies
npm install
```

```
# Start backend server
npm start
```

DOCUMENT 9: VIDEO DEMO SCRIPT & SCREEN RECORDING GUIDE

(Refer to Layer-by-Layer script established for Team Lead Himesh Roshan M J)

1. **Layer 1: Project Setup:** Execute npm start in VS Code terminal. Show server listening on Port 5000 and MongoDB connection verified.
2. **Layer 2: Auth Verification:** Execute POST /api/auth/login in Postman. Copy access token.
3. **Layer 3: Material Storage:** Execute POST /api/materials/upload with test_material.txt. Copy document _id.
4. **Layer 4: AI Generation:** Execute POST to /summarize, /quiz, and /study-plan sequentially,

highlighting generated bullet points, options, and schedules.

DOCUMENT 10: FINAL COMPREHENSIVE PROJECT REPORT

10.1 Abstract

AI-STUDYBUDDY is a specialized learning assistant backend built on the MERN stack. It leverages artificial intelligence to parse academic text and automatically synthesize bulleted summaries, practice quizzes, flashcards, and personalized study routines. The system enforces strict JWT authentication, uses Mongoose models for reliable MongoDB data management, and connects seamlessly to Google Gemini APIs.

10.2 Conclusion & Future Scope

The backend core for AI-STUDYBUDDY is completely implemented, verified, and ready for deployment. All primary objectives—including authentication, file ingestion, and multi-mode AI generation—have been fulfilled.

Future Expansion Roadmap:

- **Frontend UI:** Development of a React.js client interface.
- **PDF & OCR Support:** Expansion of Multer pipelines to extract text directly from scanned PDF lecture slides using OCR.
- **Analytics Engine:** Visual performance tracking for student quiz scores over time.